

CODE :

```
#include<stdio.h>

#include<string.h>

#include<stdbool.h>

#include<stdlib.h>


#ifdef _WIN32
#define strcasecmp _stricmp
#endif


#define MAX_STUDENT 1000
#define FILENAME "student.dat"


typedef struct{
    char name[30];
    int id;
    char department[50];
    float cgpa;
}student;


student students[MAX_STUDENT];
int studentCount=0;


bool running=true;
void loadFromFile();
void saveToFile();
void Menu();
void addStudents();
```

```
void viewAllStudents();
int searchByName(char student_name[30]);
int searchById(int student_id);
int searchByDepartment(char student_department[50]);
void updateStudent(int Index);
void deleteStudent(int Index);
void sortByIdAsc();
void sortByIdDes();
void goBackorEXit();
void clearInputBuffer();
void exitProject();
```

```
int compareByIdAsc(const void *a,const void *b);
int compareByIdDes(const void *a,const void *b);
int compareByCGPAAsc(const void *a,const void *b);
int compareByCGPADes(const void *a,const void *b);
int compareByNameAZ(const void *a,const void *b);
int compareByNameZA(const void *a,const void *b);
void clearScreen();
```

```
int main(){
    loadFromFile();
    int option;
    while(running){
        Menu();
        scanf("%d",&option);
        clearInputBuffer();
        switch(option){
            case 0:
```

```

        running=false;

        exitProject();

        break;
case 1:
    clearScreen();

    printf("\n\t\t **** Add A New Student **** \n\n");

    addStudents();

    goBackorEXit();

    break;
case 2:
    clearScreen();

    printf("\n\t\t **** All Students **** \n\n");

    viewAllStudents();

    goBackorEXit();

    break;
case 3:
    { clearScreen();

    char nam[30];

    printf("\n\t\t **** Search Students **** \n\n");

    printf(" Enter The Student Name: ");

    scanf(" %29[^\n]", nam);

    clearInputBuffer();

    int index = searchByName(nam);

    if (index >= 0) {

        printf("\n Student Found!\n");

        printf(" Name: %s\n ID: %d\n Department: %s\n\n",
            students[index].name, students[index].id,
            students[index].department,
            students[index].cgpa);
    }
}

```

```

    } else {
        printf("\n No Student Found!\n");
    }
    goBackorEXit();
    break;}
case 4:
    {
        clearScreen();
        int id;
        printf("\n\t\t **** Search Students (By ID) ****
\n\n");

        printf(" Enter Student ID: ");
        scanf("%d", &id);
        clearInputBuffer();

        int index = searchById(id);
        if (index >= 0) {
            printf("\n Student Found!\n");
            printf(" Name: %s\n ID: %d\n Department: %s\n
CGPA: %.2f\n",
                students[index].name, students[index].id,
                students[index].department,
students[index].cgpa);
        } else {
            printf("\n No Student Found!\n");
        }
        goBackorEXit();
        break;}
case 5:
    {clearScreen();

```

```

        char dep[50];
        printf("\n\t\t **** Search Students (By Department)
**** \n\n");

        printf(" Enter Department: ");
        scanf(" %49[^\n]", dep);
        clearInputBuffer();
        int count = searchByDepartment(dep);
        if (count == 0) {
            printf("\n No Student Found!\n");
        }
        goBackorEXit();
        break;
    }
case 6:
    { clearScreen();
      int id;
      printf("\n\t\t **** Edit Student **** \n\n");
      printf(" Enter Student ID: ");
      scanf("%d", &id);
      clearInputBuffer();

      int index = searchById(id);
      if (index >= 0) updateStudent(index);
      else printf("\n No Student Found!\n");
      goBackorEXit();
      break;}
case 7:
    {clearScreen();
      int id;

```

```

printf("\n\t\t **** Delete Student **** \n\n");
printf(" Enter Student ID: ");
scanf("%d", &id);
clearInputBuffer();

int index = searchById(id);
if (index >= 0) {
    char sure;
    printf(" Name: %s\n ID: %d\n Department: %s\n
CGPA: %.2f\n",
           students[index].name, students[index].id,
           students[index].department,
students[index].cgpa);
    printf("\n Are you sure you want to delete (Y/N)?
");
    scanf(" %c", &sure);
    clearInputBuffer();

    if (sure == 'Y' || sure == 'y') {
        deleteStudent(index);
        printf("\n Student Deleted Successfully!\n");
    } else {
        printf("\n Deletion Cancelled.\n");
    }
} else {
    printf("\n No Student Found!\n");
}
goBackorEXit();
break;
}

```

```
case 8:
    clearScreen();
    sortByIdAsc();
    goBackorEXit();
    break;
case 9:
    clearScreen();
    sortByIdDes();
    goBackorEXit();
    break;
case 10:
    clearScreen();
    qsort(students, studentCount, sizeof(student),
compareByCGPAAsc);
    printf("\nSorted by CGPA (Ascending):\n");
    viewAllStudents();
    goBackorEXit();
    break;
case 11:
    clearScreen();
    qsort(students, studentCount, sizeof(student),
compareByCGPADes);
    printf("\nSorted by CGPA (Descending):\n");
    viewAllStudents();
    goBackorEXit();
    break;
case 12:
    clearScreen();
    qsort(students, studentCount, sizeof(student),
compareByNameAZ);
```

```

        printf("\nSorted by Name (A to Z):\n");
        viewAllStudents();
        goBackorEXit();
        break;
    case 13:
        clearScreen();
        qsort(students, studentCount, sizeof(student),
compareByNameZA);
        printf("\nSorted by Name (Z to A):\n");
        viewAllStudents();
        goBackorEXit();
        break;
    case 14:
        clearScreen();
        saveToFile();
        printf("\nData saved successfully!\n");
        printf("Exiting program...\n");
        exitProject();
        break;
    default:
        printf("\nInvalid choice! Press Enter to
continue...");
        getchar();
    }
}
return 0;
}

void clearScreen(){
    #ifdef _WIN32
        system("cls");
    #endif
}

```



```

        #else

            system("clear");

        #endif
    }

    void loadFromFile(){

        FILE *file=fopen(FILENAME,"rb");

        if (file != NULL){

            if(fread(&studentCount, sizeof(int), 1, file) != 1) {
studentCount = 0; }

            if(fread(students, sizeof(student), studentCount, file) !=
studentCount) { studentCount = 0; }

            fclose(file);

            printf("\nData Loaded Successfully!\n\n");

        }

    }

    void saveToFile(){

        FILE *file = fopen(FILENAME, "wb");

        if(file!=NULL){

            fwrite(&studentCount,sizeof(int),1,file);

            fwrite(students,sizeof(student),studentCount,file);

            fclose(file);

        }

    }

    void Menu(){

        printf("Author:Rashadul Islam_253_16_019 \n");

        printf("\n\t\t***          DAFFODIL International
University          ***");

        printf("\n\t\t*** Department Of Computing & Information System
***\n");

        printf("\n\t\t***          Student Management System          ***\n");

```

```

printf("\t\tMain Menu:\n");
printf("\t\t=====\n");
printf("\t\t[1] Add A New Student.\n");
printf("\t\t[2] View All The Student.\n");
printf("\t\t[3] Search A Student By Using Name.\n");
printf("\t\t[4] Search A Student By Using ID.\n");
printf("\t\t[5] Search A Student By Using Department.\n");
printf("\t\t[6] Edit Student.\n");
printf("\t\t[7] Delete Student.\n");
printf("\t\t[8] Sort Students By ID(Ascending).\n");
printf("\t\t[9] Sort Students By ID(descending).\n");
printf("\t\t[10] Sort Students By CGPA(Ascending).\n");
printf("\t\t[11] Sort Students By CGPA(Descending).\n");
printf("\t\t[12] Sort Students By Name(A-Z).\n");
printf("\t\t[13] Sort Students By Name(Z-A).\n");
printf("\t\t[14] Data Save & Exit.\n");
printf("\t\t[0] Exit without Save\n");
printf("\t\t=====\n");
printf("Enter your choice:");
}

void addStudents() {
    if (studentCount >= MAX_STUDENT) {
        printf("\nStudent limit reached!\n");
        return;
    }
    int id;
    printf(" Enter ID: ");
    scanf("%d", &id);
    clearInputBuffer();

```

```

        if(searchById(id) != -1){
            printf("Student with this ID already exists!\n");
            return;
        }

        printf(" Enter Name: ");
        scanf(" %29[^\n]", students[studentCount].name);
        clearInputBuffer();

        printf(" Enter Department: ");
        scanf(" %49[^\n]", students[studentCount].department);
        clearInputBuffer();

        printf(" Enter CGPA: ");
        scanf("%f", &students[studentCount].cgpa);
        clearInputBuffer();

        students[studentCount].id = id;
        studentCount++;
        printf("\n Student Added Successfully!\n");

    }

void viewAllStudents() {
    if (studentCount == 0) {
        printf("\n No Students Available.\n");
        return;
    }

```

```

        printf("\n%-10s |%-30s |%-30s |%-10s\n", "ID", "Name", "Dept",
"CGPA");

        printf("-----\n");

        for (int i = 0; i < studentCount; i++) {
            printf("%-10d |%-30s |%-30s |%-10.2f\n",
                    students[i].id, students[i].name,
                    students[i].department, students[i].cgpa);
        }

        printf("-----\n");
    }

int searchByName(char student_name[30]) {
    for (int i = 0; i < studentCount; i++) {
        if (strcasecmp(students[i].name, student_name) == 0)
            return i;
    }
    return -1;
}

int searchById(int student_id) {
    for (int i = 0; i < studentCount; i++) {
        if (students[i].id == student_id)
            return i;
    }
    return -1;
}

```

```

int searchByDepartment(char student_department[50]) {
    int found = 0;
    for (int i = 0; i < studentCount; i++) {
        if (strcasecmp(students[i].department, student_department) ==
0) {
            printf("\n %s | ID: %d | CGPA: %.2f\n",
                students[i].name, students[i].id,
students[i].cgpa);
            found++;
        }
    }
    return found;
}

```

```

void updateStudent(int index) {
    printf("\n Editing Student: %s\n", students[index].name);

    printf(" New Name: ");
    scanf(" %29[^\n]", students[index].name);
    clearInputBuffer();

    printf(" New Department: ");
    scanf(" %49[^\n]", students[index].department);
    clearInputBuffer();

    printf(" New CGPA: ");
    scanf("%f", &students[index].cgpa);
    clearInputBuffer();
}

```

```

        printf("\n Student Updated Successfully!\n");
    }

void deleteStudent(int index) {
    for (int i = index; i < studentCount - 1; i++) {
        students[i] = students[i + 1];
    }
    studentCount--;
}

void sortByIdAsc() {
    qsort(students, studentCount, sizeof(student), compareByIdAsc);
    printf("\n Sorted by ID (Ascending):\n");
    viewAllStudents();
}

void sortByIdDes() {
    qsort(students, studentCount, sizeof(student), compareByIdDes);
    printf("\n Sorted by ID (Descending):\n");
    viewAllStudents();
}

void goBackorEXit() {
    printf("\n Press ENTER to return...");
    getchar();
    clearScreen();
}

void clearInputBuffer() {

```

```

    int c;
    while ((c = getchar()) != '\n' && c != EOF);
}

void exitProject() {
    printf("\n Exit Program By Any Press Key\n");
    exit(0);
}

// SORT FUNCTIONS

int compareByIdAsc(const void *a, const void *b) {
    return ((student *)a)->id - ((student *)b)->id;
}

int compareByIdDes(const void *a, const void *b) {
    return ((student *)b)->id - ((student *)a)->id;
}

int compareByCGPAAsc(const void *a, const void *b) {
    float x = ((student *)a)->cgpa;
    float y = ((student *)b)->cgpa;
    return (x > y) - (x < y);
}

int compareByCGPADes(const void *a, const void *b) {
    float x = ((student *)b)->cgpa;
    float y = ((student *)a)->cgpa;
    return (x > y) - (x < y);
}

```

```
int compareByNameAZ(const void *a, const void *b) {  
    return strcmp(((student *)a)->name, ((student *)b)->name);  
}
```

```
int compareByNameZA(const void *a, const void *b) {  
    return strcmp(((student *)b)->name, ((student *)a)->name);  
}
```